Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering

A report on "EduPredict - AI Student Performance Predictor"

Course Code: CSE 3200

Course Title: Software Development Project II

Authors

Mashfiq Ahmed Naushad

2103056

Section A

Supervisor

Prof. Dr. Boshir Ahmed

Professor

Dept. of CSE, RUET

Date: 09/01/2026

Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering

Certification

This is to certify that the project titled: "**EduPredict - AI Student Performance Predictor**" submitted by **Mashfiq Ahmed Naushad,** Roll Number(s) **2103056**, for the fulfillment of the requirements of the course CSE 3200 (Software Development Project II), has been examined and approved by the undersigned.

The project has been carried out under my supervision, and it is recommended for submission and evaluation.

Supervisor:
Name: _____
Designation: _____
Department: _____
Institution: _____
Date: _____

Signature: _____

Contents

	Page No.
Abstract	
Chapters	
1. Introduction	**08**
1.1 Problem Statement	**08**
1.2 Motivation	**08**
1.3 Objectives	**09**
1.4 Contribution Summary	**09**
1.5 Report Structure	**10**
2. Background & Literature Review	**12**
2.1 Theoretical Foundation	**12**
2.2 Related Work Review	**13**
2.3 Comparative Analysis	**13**
2.4 Gap Analysis	**14**
3. Requirements & Team Workflow	**15**
3.1 Functional Requirements	**15**
3.2 Non-functional Requirements	**16**
3.3 System Constraints	**16**
3.4 Technology Stack Justification	**17**
3.5 Team Structure & Individual Responsibilities	**18**
3.6 Collaboration Workflow	**18**
4. Project Management & Finance	**19**
4.1 Work Breakdown Structure (WBS)	**19**
4.2 Project Schedule — Gantt Chart	**19**
4.3 Budget & Financial Cost Analysis	**20**
4.4 Risk Analysis & Mitigation Plan	**20**
4.5 Summary of Management Decisions	**21**
5. System Design & Architecture	**22**
5.1 Methodological Structure	**22**
5.2 High-Level Architecture Diagram	**22**
5.3 Module Interaction Diagram	**23**
5.4 Database Schema / ER Diagram	**24**
5.5 Interface Specifications / API Contracts	**25**

5.6 Security & Performance Design 26
6. Implementation 27
6.1 Development Environment Setup 27
6.2 Coding Standards & Version Control Strategy 27
6.3 Module-Wise Implementation 28
6.3.x Module Ownership Block 29
6.4 System Integration Workflow 30
6.5 Performance, Optimization, and Security Techniques 31
7. Testing, Results & Evaluation 32
7.1 Testing Strategy 32
7.2 Test Case Tables 32
7.3 Performance Evaluation 33
7.4 Comparison with Baseline or Existing Systems 33
7.5 Screenshots & Execution Results 34 - 42
8. Discussion & Analysis 43
8.1 Interpretation of Findings 43
8.2 Strengths of the Project 43
8.3 Limitations 43
8.4 Recommendations for Future Development 44
8.5 Reflection on Design & Implementation Decisions 44
9. Life-Long Learning impact 45
9.1 Technical & Professional Skills Acquired 45
9.2 Learning New Technologies & Tools 45
9.3 Future Growth & Directions 45
10. Conclusion 46
10.1 Summary of Work 46
10.2 Achievement of Objectives 46
10.3 Final Remarks 46

REFERENCES 47

APPENDICES 47

Abstract

Student performance monitoring in many educational institutions remains largely reactive, where students facing academic difficulties are identified only after a significant decline in grades or overall performance. This delayed recognition often reduces the effectiveness of timely intervention. ***EduPredict*** *is a* ***web-based*** *student performance monitoring system designed to address this challenge by predicting whether a student is At-Risk or Not-At-Risk using commonly available academic indicators such as attendance, assignment performance, quizzes, examinations, and GPA.*

EduPredict *offers a secure, role-based platform tailored for Administrators, Teachers, and Students. The system uses* ***JWT-based*** *authentication combined with server-enforced* ***Role-Based Access Control*** *(RBAC) to ensure data security, privacy, and controlled access to academic records. Key functionalities include structured academic record management, performance tracking, and reporting tools that enable stakeholders to monitor progress and identify potential risks early.*

At the core of ***EduPredict*** *is a machine learning pipeline built using an ensemble of* ***Logistic Regression*** *and* ***LightGBM*** *models. This combination allows the system to balance interpretability with predictive accuracy, capturing both linear trends and complex patterns in student data. To enhance transparency and trust, the system integrates SHAP* ***TreeExplainer****, enabling educators to understand how individual features contribute to each prediction. Additionally, a lightweight model registry supports model versioning, evaluation, and promotion, ensuring reproducibility and maintainability.*

The backend of the system is implemented using ***FastAPI****, asynchronous* ***SQLAlchemy****, and* ***Alembic****, providing scalability and efficient database management. The frontend is developed with* ***Next.js*** *(App Router),* ***TypeScript****, and* ***Tailwind*** *CSS, offering a modern and responsive user interface. Overall,* ***EduPredict*** *delivers an end-to-end, deployable, and interpretable solution that supports early identification of academic risk and enables proactive, data-driven intervention by educators.*

List of Figures

- *Figure 1: High-Level Architecture Diagram*
- *Figure 2: Module Interaction Diagram*
- *Figure 3: Database Schema / ER Diagram*
- *Figure 4: Sample Prediction + Explanation Output Screen*
- *Figure 5: User Authentication and Login Interface*
- *Figure 6: Admin Dashboard: User Management and Creation*
- *Figure 7: Teacher Workflow: Academic Record Entry*
- *Figure 8: Student View: Individual Performance Tracking*
- *Figure 9: ML Model Management: Versioning and Promotion UI*
- *Figure 10: Backend Execution Logs and Successful Training Output*

List of Tables

- *Table 1: Functional Requirements*
- *Table 2: Non-functional Requirements*
- *Table 3: Work Breakdown Structure*
- *Table 4: Risk Analysis & Mitigation Plan*
- *Table 5: API Contract Summary*
- *Table 6: Test Case Table*

Chapter 1
Introduction

1.1 Problem Statement

In most academic environments, identifying students who are at risk of failing or significantly underperforming is often done after midterm/final results or after repeated poor assessments. This makes intervention late and reduces the chance of improvement. The project addresses the problem of building an automated, secure, and interpretable system that can:

- Collect and maintain structured academic indicators (attendance, assignment scores, quiz scores, exam scores, GPA).
- Predict academic risk early (At-Risk vs Not-At-Risk).
- Provide explanations to justify the prediction so educators can act with confidence.
- Ensure privacy and data protection using strong authentication and role-based authorization.

In short, the system solves the lack of a unified platform that combines academic record management, early risk prediction, and explainable results while enforcing real-world access control.

1.2 Motivation

The motivation of EduPredict comes from the real needs of educational institutions:

- Early identification enables timely mentoring, tutoring, counseling, or academic support.
- Manual tracking by teachers/admins is time-consuming and inconsistent.
- A data-driven system can help teachers focus on high-risk students efficiently.
- Explainable predictions are essential because educators must understand “why” a student is considered at risk before taking action.
- Security is critical because academic information is sensitive; the system must prevent unauthorized access and misuse.

1.3 Objectives

The measurable goals of EduPredict are:

- Develop a full-stack web application with separate role-based flows for Admin, Teacher, and Student.
- Implement secure authentication using JWT tokens and refresh tokens.
- Enforce Role-Based Access Control (RBAC) server-side for all protected routes.
- Support academic record CRUD operations with correct permissions:
    - Teachers/Admins create and update records,
    - Admins delete records,
    - Students view only their own records.
- Integrate a machine learning pipeline to:
    - Train a model from stored academic records,
    - Predict a risk probability and class label,
    - Provide local explanations (feature contributions) for predictions.
- Maintain a model registry that supports:
    - Versioning of trained models,
    - Listing available versions,
    - Promoting a specific version to “latest”.
- Provide reliable local setup using SQLite and optional Postgres using Docker.

1.4 Contribution Summary

EduPredict contributes an end-to-end solution with the following key components:

Backend Contributions (FastAPI)

- **Auth module:**
    - Student registration endpoint.
    - OAuth2 password login endpoint returning access + refresh tokens.
    - Refresh endpoint implementing refresh-token rotation.
    - Logout endpoint revoking refresh tokens.
    - Current user profile endpoint (/me).
- **RBAC module:**
    - Server-enforced roles (admin/teacher/student).
    - Permission checks on each router to protect resources.

- **Academic module:**
  - Academic record data model with validation constraints (0–100 for percentages, 0.0–4.0 for GPA).
  - Teacher/Admin record creation and update.
  - Admin-only deletion.
  - Student-only access to self records.
- **ML module:**
  - Predict endpoint returning probability, classification, confidence, threshold, and model version.
  - Explain endpoint returning top-k feature contributions with direction and magnitude.
  - Train endpoint (admin-only) that trains a new model from academic_records data.
- **Model registry:**
  - Stores model artifacts (joblib) and metadata (json).
  - Maintains a LATEST pointer and allows model promotion.

Frontend Contributions (Next.js)

- Modern UI with Next.js App Router + TypeScript + Tailwind CSS.
- Configurable backend base URL.
- UI flow designed for role-based usage (admin dashboards, teacher record management, student viewing and prediction).

Machine Learning Contributions

- Ensemble approach using Logistic Regression and LightGBM.
- Explainability using SHAP TreeExplainer on the LightGBM component.
- Model versioning with reproducible metadata and feature names.

1.5 Report Structure

This report is organized as follows:

- **Chapter 2** explains background concepts and reviews related work in academic risk prediction and explainable ML.
- **Chapter 3** lists system requirements (functional and non-functional), constraints, technology justification, and workflow.

- **Chapter 4** explains project planning, schedule, budget, and risk management decisions.
- **Chapter 5** describes the architecture, module interactions, database schema, and API contracts.
- **Chapter 6** explains how the system was implemented module-by-module, including development setup and standards.
- **Chapter 7** presents testing approach, test cases, and evaluation discussion.
- **Chapter 8** analyzes results, strengths, limitations, and future improvements.
- **Chapter 9** summarizes skills and learning outcomes from building the project.
- **Chapter 10** provides the conclusion, objective achievement, and final remarks.

Chapter 2
Background & Literature Review

2.1 Theoretical Foundation

EduPredict is based on concepts from software engineering, data science, and machine learning:

Binary Classification

- The target is to classify students into two classes: At-Risk (1) or Not-At-Risk (0).
- The model outputs a probability `p(y=1|x)` for a student feature vector `x`.

Logistic Regression

- A simple and interpretable baseline classifier.
- Provides stable results for tabular features and can generalize well with limited data.

Gradient Boosted Trees (LightGBM)

- Captures non-linear feature interactions and complex patterns.
- Efficient training and strong performance on structured/tabular datasets.

Ensemble Learning

- Combining multiple models often improves stability and generalization.
- In EduPredict, an ensemble artifact combines predictions from Logistic Regression and LightGBM.

Explainable AI (XAI) and SHAP

- SHAP provides feature-attribution values that explain the contribution of each feature.
- TreeExplainer provides fast explanations for tree-based models like LightGBM.
- Explanations improve trust and support decision-making by educators.

Secure Web Systems

- JWT-based authentication for stateless API access.
- Refresh token mechanism for secure long-lived sessions.
- RBAC for access control and privacy.

2.2 Related Work Review

Academic risk prediction is widely studied in Educational Data Mining (EDM). Common approaches include:

- Rule-based early warning systems (e.g., attendance below a threshold).
- Statistical models predicting failure probability.
- Machine learning models using attendance, assessment marks, and GPA.
- Dashboard systems for teachers and academic advisors.

However, many solutions focus only on accuracy and ignore:

- Explainability (why the model predicted risk).
- Role-based data privacy (students vs teachers vs admins).
- Practical system deployment with authentication, database, and API design.

2.3 Comparative Analysis

Manual Monitoring

- *Strength:* Direct teacher insight.
- *Weakness:* Time-consuming, inconsistent, late detection.

Rule-Based Thresholding

- *Strength:* Easy to understand and implement.
- *Weakness:* Ignores interactions (e.g., good GPA but very poor attendance) and can be inaccurate.

ML-Based Prediction Without Explainability

- *Strength:* Higher accuracy potential.
- *Weakness:* “Black box” risk, low trust, difficult to justify interventions.

EduPredict's Approach

- Combines ML with explainability (SHAP) and operational features (RBAC, CRUD, model registry).
- Designed to be practical for real institutional stakeholders.

2.4 Gap Analysis

The gaps addressed by EduPredict are:

- Lack of integrated platforms that combine academic data management, prediction, and explainability.
- Lack of strict RBAC implementation in many academic prediction prototypes.
- Lack of model lifecycle management (versioning, promotion, reproducibility).
- Limited deployment readiness in many academic ML systems.

EduPredict fills these gaps with a full-stack, secure, explainable, versioned ML platform.

Chapter 3

Requirements & Team Workflow

3.1 Functional Requirements

ID	Category	Requirement Description
FR1	Auth	The system shall allow student registration.
FR2	Auth	The system shall allow user login using email/password and return JWT access + refresh tokens.
FR3	Auth	The system shall allow token refresh (refresh token rotation).
FR4	Auth	The system shall allow logout and revoke refresh tokens.
FR5	Auth	The system shall provide a `/me` endpoint to return current user identity.
FR6	Auth	The system shall enforce RBAC for all protected endpoints.
FR7	User Mgmt	Admin shall list users.
FR8	User Mgmt	Admin shall create users (teacher/admin/student).
FR9	User Mgmt	Admin shall update users (role, activation status).
FR10	Academics	Teacher/Admin shall create academic records for students.
FR11	Academics	Teacher/Admin shall update academic records.
FR12	Academics	Admin shall delete academic records.
FR13	Academics	Student shall view only their own academic records.
FR14	Academics	Teacher/Admin shall list/view academic records according to access rules.
FR15	ML	The system shall provide prediction (raw feature, record ID, or latest record).
FR16	ML	The system shall provide explanation with top-k factors for a prediction.
FR17	ML	Admin shall train a new model from stored academic records.
FR18	ML	Admin shall list model versions and view model metadata.
FR19	ML	Admin shall promote a specific model version to be the latest active model.
FR20	System	The system shall provide a health endpoint.
FR21	System	The system shall provide documentation via OpenAPI (Swagger UI).

Table 1: Functional Requirements

3.2 Non-functional Requirements

ID	Category	Requirement Description
NFR1	Security	Passwords must be stored as hashes, not plain text.
NFR2	Security	JWT validation must be enforced on protected routes.
NFR3	Security	RBAC must be enforced server-side to prevent unauthorized access.
NFR4	Reliability	The system must provide clear error messages (e.g., no model available yet).
NFR5	Maintainability	Code must be modular (routers, services, models, schemas, ml).
NFR6	Reliability	Database migrations should ensure consistent schema evolution.
NFR7	Performance	Prediction should be fast for single-record requests.
NFR8	Performance	Explanation should be feasible on standard hardware (TreeExplainer).
NFR9	Performance	Model loading should be cached to avoid repeated disk reads.
NFR10	Usability	UI should be responsive and easy to use.
NFR11	Portability	The system should support local development on Windows.
NFR12	Portability	The system should support SQLite by default and Postgres via Docker.

Table 2: Non-functional Requirements

3.3 System Constraints

- Limited dataset availability can reduce model quality; training requires minimum rows.
- Sensitive student academic data requires strict privacy.
- Explainability computations add overhead compared to prediction.
- Training endpoint is synchronous; large datasets may require background processing.
- Deployment environment must manage environment variables correctly (DB URL, JWT secrets, CORS).

3.4 Technology Stack Justification

Frontend (Next.js + TypeScript + Tailwind CSS)

- Next.js App Router supports modern routing and UI composition.
- TypeScript improves maintainability and reduces runtime bugs.
- Tailwind CSS provides fast, consistent, responsive UI styling.
- Additional UI libraries (Radix UI) support accessible components.

Backend (FastAPI + async SQLAlchemy + Alembic)

- FastAPI provides high performance, easy endpoint creation, and automatic OpenAPI docs.
- Async SQLAlchemy supports scalable DB access.
- Alembic manages schema migrations reliably.

Database (SQLite + Postgres)

- SQLite enables quick local setup without external services.
- Postgres provides production-ready relational storage; Docker Compose makes it easy to run locally.

ML Stack (scikit-learn + LightGBM + SHAP)

- scikit-learn supports pipelines (StandardScaler + Logistic Regression).
- LightGBM provides strong performance on tabular data.
- SHAP TreeExplainer provides fast feature-attribution explanations.

Model Registry (file-based)

- A lightweight registry provides versioning without external infrastructure.
- Joblib + JSON metadata makes artifacts portable and reproducible.

3.5 Team Structure & Individual Responsibilities

This project was developed by a single author.

Mashfiq Ahmed Naushad (2103056, Section A)

- Designed system requirements and architecture.
- Implemented backend API (auth, RBAC, academic CRUD, ML endpoints).
- Designed database schema and wrote migrations (Alembic).
- Implemented ML pipeline (training, inference, explainability).
- Implemented model registry (versioning, metadata, promotion).
- Integrated backend and frontend configuration.
- Wrote tests and validated endpoints.
- Prepared documentation and project report.

3.6 Collaboration Workflow

Since this was a single-developer project, collaboration workflow focused on disciplined development practices:

- Git-based development with meaningful commits after completing logical modules (auth, academics, ML, registry).
- Modular folder structure to reduce coupling and keep changes localized.
- Regular local testing using pytest to confirm correctness after changes.
- Use of environment example files to standardize configuration for different setups (local SQLite vs Docker Postgres).
- For task tracking, a simple checklist approach was used (requirements → backend → ML → frontend → testing → documentation).

Chapter 4
Project Management & Finance

4.1 Work Breakdown Structure (WBS)

WBS ID	Phase	Tasks
1.0	Requirement Analysis	Identify stakeholders, Define FRs/NFRs, Define features.
2.0	Backend Foundation	FastAPI scaffold, Settings, CORS, Health endpoint.
3.0	Database & Migrations	Schema design, SQLAlchemy models, Alembic migrations.
4.0	Authentication & RBAC	Password hashing, JWT tokens, Refresh token rotation, RBAC.
5.0	Academic Records	CRUD endpoints, RBAC rules, Input validation.
6.0	Machine Learning Pipeline	Training, Registry, Inference, Explanation (SHAP).
7.0	Frontend Development	Next.js setup, UI pages, API integration.
8.0	Testing & Documentation	Integration tests, Regression testing, Final report.

Table 3: Work Breakdown Structure

4.2 Project Schedule — Gantt Chart

A representative schedule (adjust weeks/dates if required):

Week 1: Requirement analysis, role definition, feature selection. **Week 2:** Backend scaffold, settings, DB models, migrations. **Week 3:** Auth module, JWT + refresh tokens, RBAC enforcement. **Week 4:** Academic record CRUD and permissions, validation constraints. **Week 5:** ML training pipeline, registry, inference, explanation. **Week 6:** Frontend integration, end-to-end testing, documentation and report.

4.3 Budget & Financial Cost Analysis

The project is designed to operate on minimal cost:

Development cost:

- Local laptop/PC, internet connectivity (no direct purchased services required).

Software cost:

- All major tools and libraries used are open-source (FastAPI, SQLAlchemy, Alembic, Next.js, LightGBM, scikit-learn, SHAP).

Infrastructure cost:

- Local DB: SQLite (free)
- Optional local Postgres via Docker Compose (free)
- Optional deployment on free tiers (frontend + backend + database) can be done without paid plans.

Estimated recurring cost:

- BDT 0 on free tiers (excluding internet/electricity).

4.4 Risk Analysis & Mitigation Plan

Risk	Impact	Likelihood	Mitigation Strategy
Insufficient training data	High	Medium	Use demo dataset; enforce minimum rows.
Privacy leak / Access control	High	Low	Server-side RBAC; test access scenarios.
Model misinterpretation	Med	Medium	Provide SHAP explanations; configurable threshold.

Library dependency changes	Med	Medium	Pin library versions; handle output formats.
Deployment misconfiguration	Med	Medium	Provide .env.example; document vars.
Sync training blocking server	Med	Low	Restrict to admin; future async job queue.

Table 4: Risk Analysis & Mitigation Plan

4.5 Summary of Management Decisions

- Adopted a modular architecture to keep components independent (routers/services/ml/models).
- Implemented DB migrations early to ensure stable schema evolution.
- Implemented security (auth + RBAC) early to avoid later redesign.
- Used a file-based model registry for simplicity and reproducibility.
- Added automated tests to validate authorization rules and ML endpoints.

Chapter 5
System Design & Architecture

5.1 Methodological Structure

EduPredict follows a layered architecture:

- **Presentation Layer:** Next.js frontend UI
- **API Layer:** FastAPI routers and request validation
- **Business Logic Layer:** Services and RBAC dependencies
- **Data Layer:** SQLAlchemy async ORM + database
- **ML Layer:** Training, inference, explainability modules
- **Model Lifecycle Layer:** Model registry with versioning and metadata

Development followed an iterative build process: Foundation → DB → Auth/RBAC → Academics → ML → Frontend integration → Testing → Documentation

5.2 High-Level Architecture Diagram

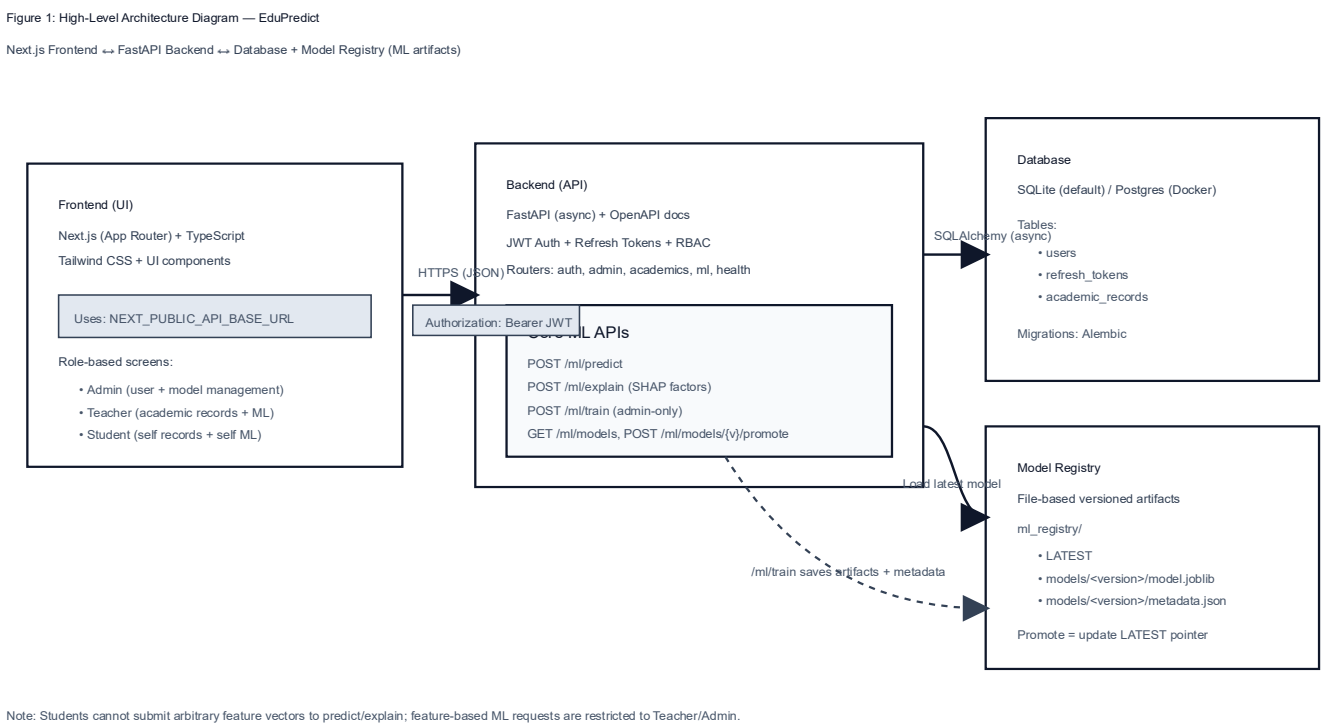


Figure 1: High-Level Architecture Diagram

- The frontend communicates with the backend via HTTP requests.
- The backend authenticates users using JWT tokens.
- The backend reads/writes academic data to the database (SQLite/Postgres).
- The backend reads ML artifacts from the model registry directory.
- Prediction/explanation results are returned to the frontend for display.

5.3 Module Interaction Diagram

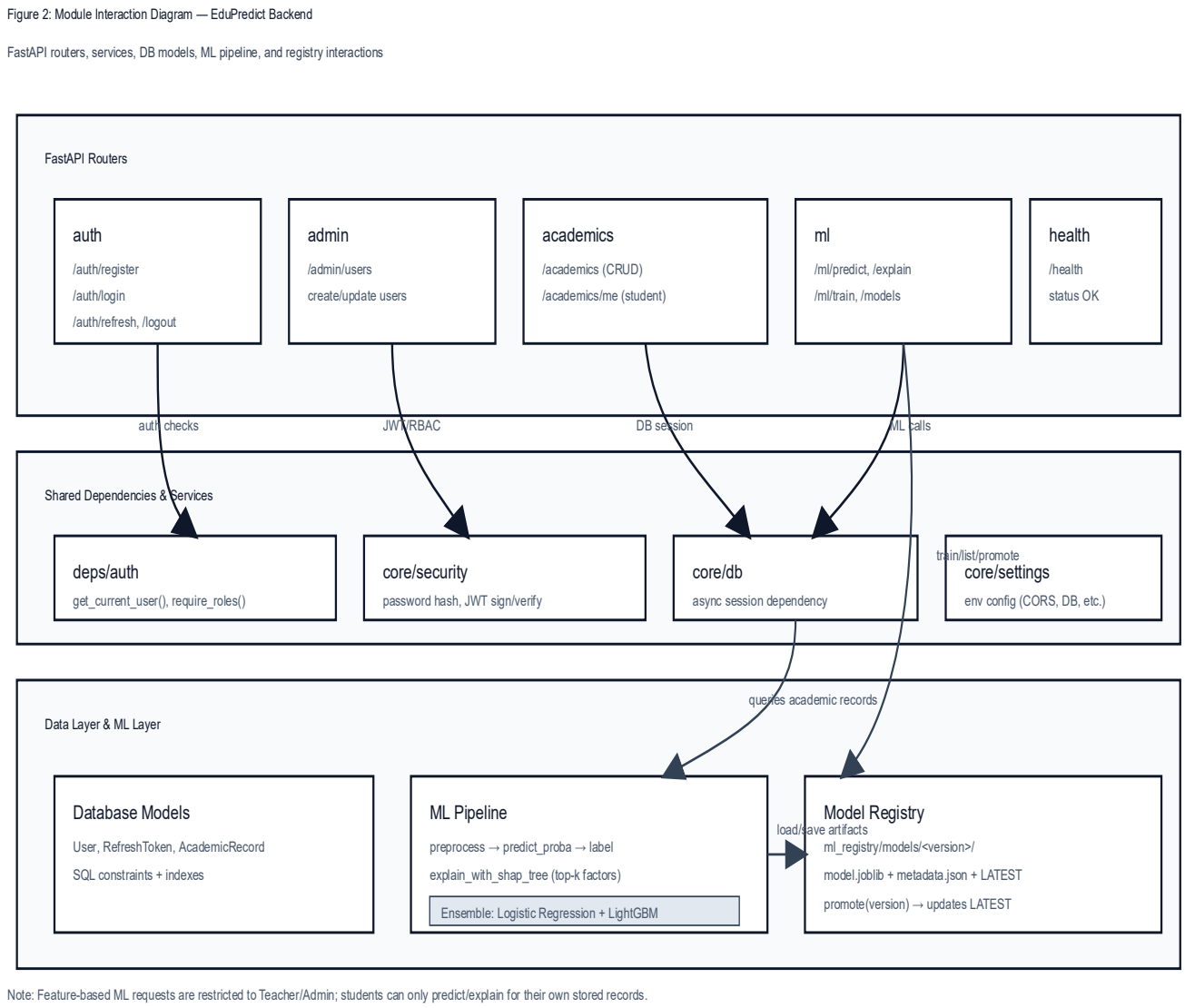


Figure 2: Module Interaction Diagram

Frontend

- Login → receives tokens → stores and sends JWT
- Academic pages → call /academics endpoints
- ML pages → call /ml/predict and /ml/explain
- Admin pages → call /admin endpoints and model management endpoints

Backend Routers

- /auth → security + users + refresh_tokens
- /academics → academic_records + RBAC checks
- /ml → preprocessing + inference + explain + registry
- /admin → user management + RBAC checks
- /health → health status

ML Pipeline Interaction

- /ml/train reads academic_records → trains model → saves to registry → updates LATEST
- /ml/predict loads latest model → preprocesses features → returns probability + label
- /ml/explain loads latest model → SHAP explanation → returns top-k factor list

5.4 Database Schema / ER Diagram

Figure 3: Database Schema / ER Diagram

Entities:

- Users (admin/teacher/student)
- RefreshTokens (many per user, revocable)
- AcademicRecords (many per student)

Relationships:

- One User → Many RefreshTokens
- One User (student) → Many AcademicRecords

Important constraints:

- Academic indicators validated (0–100 for percentages, 0.0–4.0 for GPA)
- Unique email for users
- Indexes on email, role, student_user_id, token identifiers

5.5 Interface Specifications / API Contracts

Method	Endpoint	Description	Access Level
GET	/health	System health check	Public
POST	/auth/login	Login with email/password	Public
POST	/auth/refresh	Refresh access token	Public (valid refresh token)
POST	/auth/logout	Revoke refresh token	Authenticated
GET	/admin/users	List all users	Admin
GET	/academics	List academic records	Teacher/Admin (All), Student (Self)
POST	/academics	Create academic record	Teacher/Admin
DELETE	/academics/{id}	Delete academic record	Admin

POST	/ml/predict	Predict risk probability	Authenticated (Role restricted)
POST	/ml/explain	Explain prediction (SHAP)	Authenticated (Role restricted)
POST	/ml/train	Train new model	Admin

Table 5: API Contract Summary

5.6 Security & Performance Design

Security Design

- Password hashing for stored credentials.
- JWT access tokens for API calls.
- Refresh tokens stored and revocable in DB with expiry control.
- RBAC enforced on the backend (not trusting frontend).
- Students restricted from feature-based prediction/explanation to prevent manipulation.
- CORS configured via settings to allow frontend origin(s) safely.

Performance Design

- Model loading cached in memory to avoid frequent disk reads.
- Prediction operates on a single-row dataframe for fast inference.
- Explanation uses TreeExplainer for efficiency.
- DB indexes improve user lookup and record query performance.

Chapter 6
Implementation

6.1 Development Environment Setup

Operating System: Windows

Backend Environment:

- Python virtual environment (.venv)
- Dependencies managed via pyproject.toml
- Run migrations using Alembic
- Run API using Uvicorn
- **Database:**
  - SQLite for local development (default)
  - Optional Postgres using Docker Compose (postgres:16)

Frontend Environment:

- Node.js environment
- Dependencies managed via package.json
- Next.js dev server for local UI development

6.2 Coding Standards & Version Control Strategy

Coding Standards

- Python style: consistent formatting, clear function naming, modular structure.
- Type hints used in Python for clarity and maintainability.
- Lint configuration available (Ruff) for code quality.

Version Control Strategy

- Git-based version control.
- Feature-based commits (auth, academics, ML, registry, frontend).
- Regular testing after major changes to avoid regression.

6.3 Module-Wise Implementation

6.3.1 Authentication & RBAC Module

Implementation details:

- User model includes role enum (admin/teacher/student).
- Login uses OAuth2 password flow to issue access token + refresh token.
- Refresh tokens are stored in a `refresh_tokens` table (hash + expiry + revocation).
- RBAC implemented as dependencies that check current user role before allowing endpoint access.
- Critical rule: students are restricted to their own data; teachers/admins get broader privileges.

Module Ownership Block

- **Developer:** Mashfiq Ahmed Naushad (2103056)
- **Tasks completed:**
  - User schema and role enum
  - JWT auth flow, refresh token rotation, logout revocation
  - RBAC dependencies and enforcement on routers
- **Git commit evidence:** (Insert commit IDs/screenshots if required)

6.3.2 Academic Records Module

Implementation details:

- `AcademicRecord` stores `attendance_pct`, `assignments_pct`, `quizzes_pct`, `exams_pct`, `gpa`, `term`.
- DB constraints enforce valid numeric ranges.
- **Endpoints implement:**
  - Teacher/Admin creation and update
  - Admin-only deletion
  - Student-only access to own records
- Query patterns include filtering by `student_user_id` and ordering by timestamps.

Module Ownership Block

- **Developer:** Mashfiq Ahmed Naushad (2103056)
- **Tasks completed:**
  - `AcademicRecord` model + migrations
  - CRUD endpoints with RBAC enforcement
  - Validation and constraints
- **Git commit evidence:** (Insert commit IDs/screenshots if required)

6.3.3 Machine Learning Module (Training, Inference, Explainability)

Implementation details:

- **Training**
  - Reads academic_records feature columns.
  - Splits into train/test (default test_size 0.2).
  - Trains Logistic Regression (with StandardScaler) and LightGBM.
  - Combines into an ensemble artifact for inference.
  - Evaluates on test set and stores metrics in metadata.
  - Generates unique UTC version string with microseconds.
- **Inference**
  - Loads latest model version from registry.
  - Preprocesses records and returns probability p.
  - Outputs classification label based on configurable threshold.
- **Explainability**
  - Uses SHAP TreeExplainer on LightGBM model.
  - Produces per-feature contributions and directions (increase/decrease risk).
  - Returns top-k absolute contributions.
- **Model Registry**
  - Stores model.joblib and metadata.json under version directories.
  - Maintains a LATEST pointer.
  - Supports list_versions and promote operations.

Module Ownership Block

- **Developer:** Mashfiq Ahmed Naushad (2103056)
- **Tasks completed:**
  - Training pipeline and evaluation
  - Inference endpoint logic and caching
  - SHAP explanation output formatting
  - Model registry versioning and promotion
- **Git commit evidence:** (Insert commit IDs/screenshots if required)

6.3.4 Frontend Module (Next.js UI)

Implementation details:

- Next.js App Router structure for pages.
- Tailwind CSS for responsive styling.
- API base URL configured via NEXT_PUBLIC_API_BASE_URL.
- **Role-based navigation and screens for:**
  - Authentication
  - Academic record views and management
  - Prediction and explanation display
  - Admin user management and model management pages (as applicable)

Module Ownership Block

- **Developer:** Mashfiq Ahmed Naushad (2103056)
- **Tasks completed:**
  - UI structure and styling
  - API integration patterns using JWT
  - Display of predictions and explanations
- **Git commit evidence:** (Insert commit IDs/screenshots if required)

6.4 System Integration Workflow

Integration steps:

1. Configure backend environment variables (.env) and run migrations.
2. Start backend API server and verify /docs and /health.
3. Configure frontend .env.local with backend base URL.
4. Start Next.js dev server.
5. **Validate end-to-end:**
   - Register/login
   - Role restrictions (student vs teacher vs admin)
   - Academic record creation and retrieval
   - Model training (admin) then predict/explain from UI/API

6.5 Performance, Optimization, and Security Techniques

Performance techniques:

- Caching loaded model artifact (in-memory) to avoid repeated loads.
- Efficient single-row prediction.
- TreeExplainer for fast SHAP computation.

Optimization techniques:

- Database indexes for frequent lookups (email, role, foreign keys).
- Clean tabular schema to reduce preprocessing complexity.

Security techniques:

- Password hashing and secure token handling.
- Refresh token rotation and revocation.
- Strict RBAC checks on every sensitive endpoint.
- Prevent students from submitting arbitrary features for predictions.

Chapter 7
Testing, Results & Evaluation

7.1 Testing Strategy

Testing types used:

- Integration tests for API endpoints (auth flow, RBAC rules, academics module).
- **ML endpoint tests covering:**
  - Training
  - Prediction
  - Explanation
  - Model listing and version behavior

Tools:

- pytest and pytest-asyncio for async endpoint tests.

Validation approach:

- Test role-specific behavior (student cannot access other students, teacher cannot do admin-only actions).
- Confirm that ML endpoints return expected schema and handle missing-model conditions properly.

7.2 Test Case Tables

Table 6: Sample Test Cases

ID	Test Case	Input	Expected Output	Actual Output	Status
T1	Registration	Valid email/password	User created with student role	User created	Passed
T2	Login	Valid credentials	Access token + refresh token	Access token returned	Passed
T3	Refresh token	Valid refresh token	New access token + rotated refresh token; old token invalidated	Tokens refreshed	Passed
T4	RBAC student record access	Student requests another student's record	403 Forbidden	403 Forbidden	Passed

T5	Teacher creates academic record	Teacher submits record for student	200 OK, record created	Record created	Passed
T6	ML predict (record-based)	academic_record_id	Probability + label + model_version	Correct prediction	Passed
T7	ML explain	academic_record_id or student_user_id	Top-k factors with direction and impact	List of factors returned	Passed
T8	Admin trains model	Train request with enough rows	New model version created and stored	Version stored	Passed

7.3 Performance Evaluation

Formal benchmarking (latency/throughput) was not performed as a separate experimental study. However, the design supports practical performance:

- Prediction is lightweight because it processes one row at a time.
- Model artifact loading is cached to avoid repeated disk IO.
- SHAP explanation is heavier than prediction but uses TreeExplainer for efficiency.

Suggested evaluation method (if required):

- Measure average response time for /ml/predict and /ml/explain over N requests.
- Record CPU and memory usage during explanation and training.
- Evaluate model accuracy metrics from stored metadata after training.

7.4 Comparison with Baseline or Existing Systems

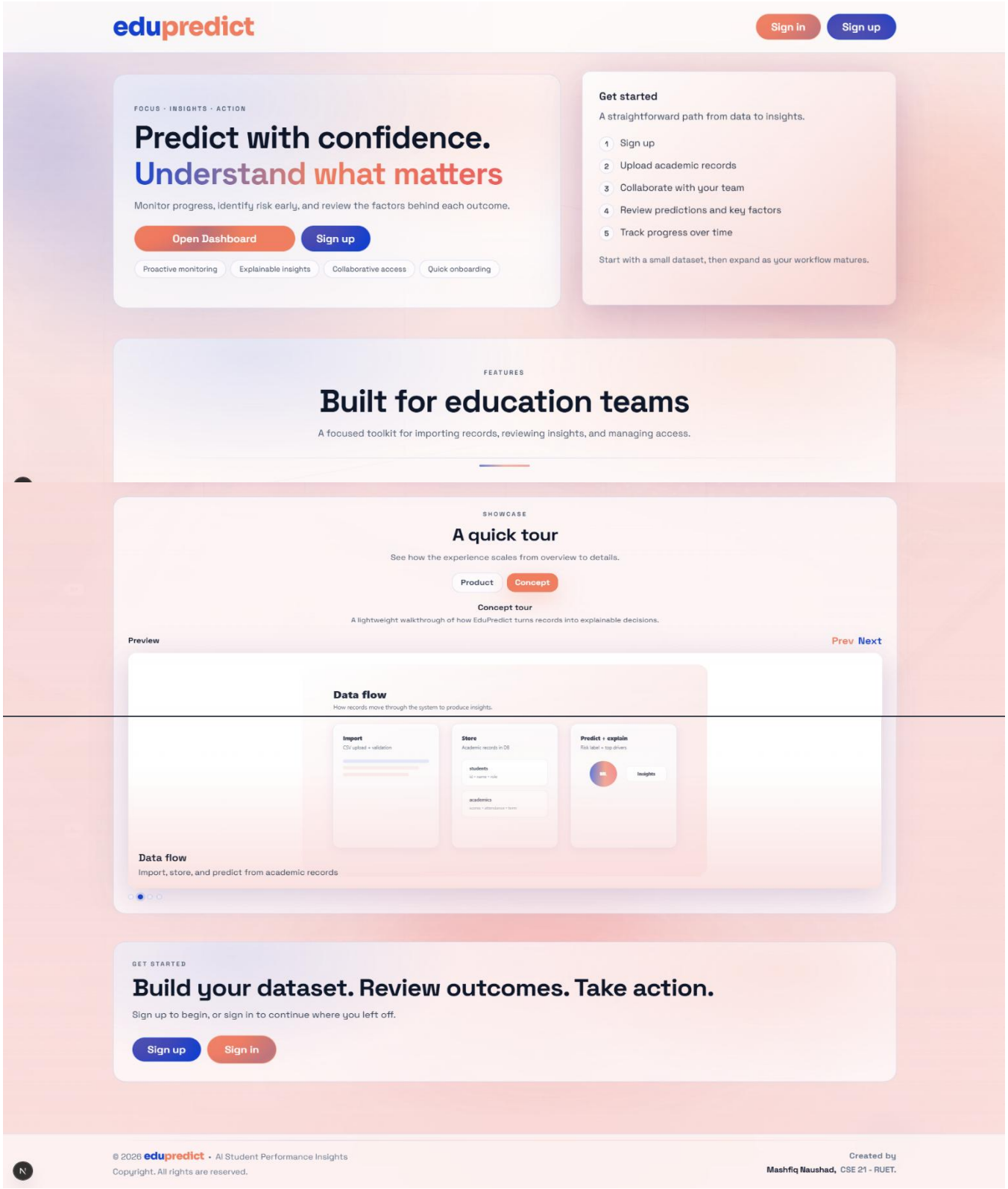
Baseline: Rule-based threshold system (e.g., attendance < 60% implies risk).

Comparison:

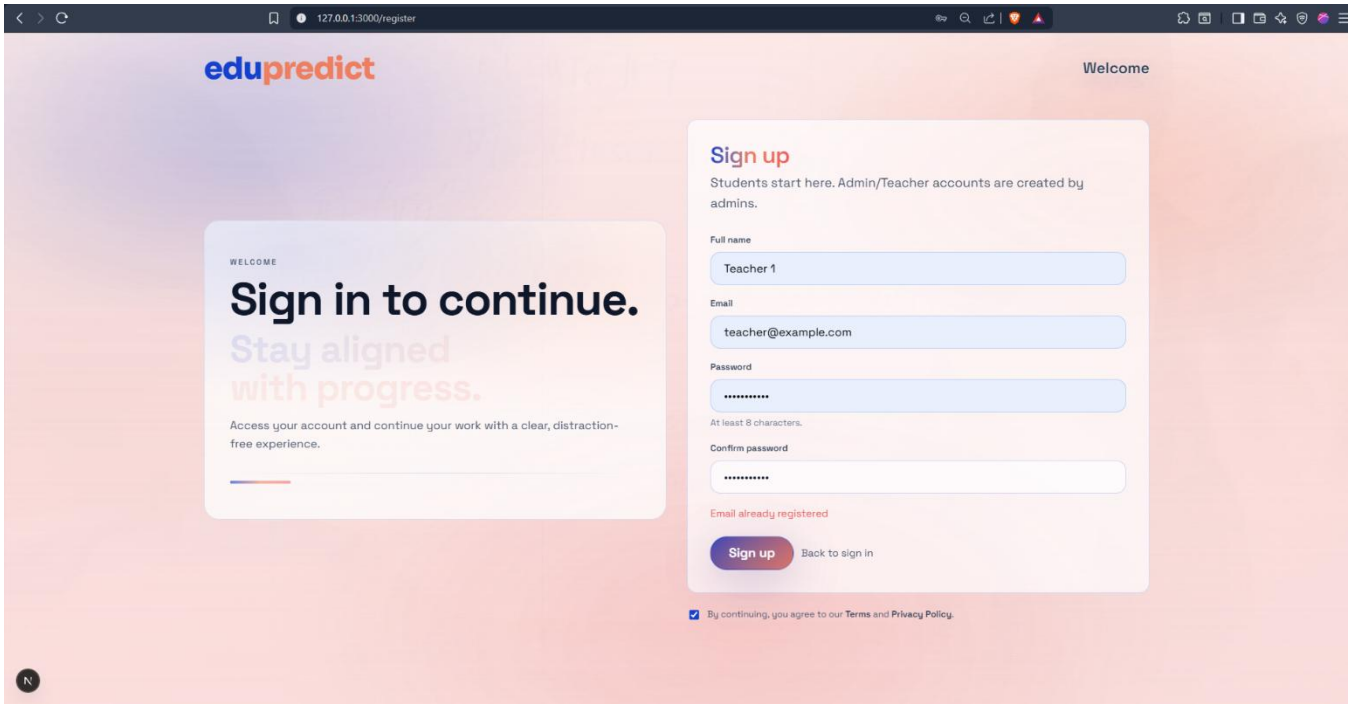
- Baseline is simple but often inaccurate because it ignores interactions between features.
- EduPredict uses ML (Logistic Regression + LightGBM) to incorporate multiple indicators simultaneously and capture both linear and non-linear patterns.
- EduPredict adds explainability (SHAP) so the results are not opaque like typical black-box systems.

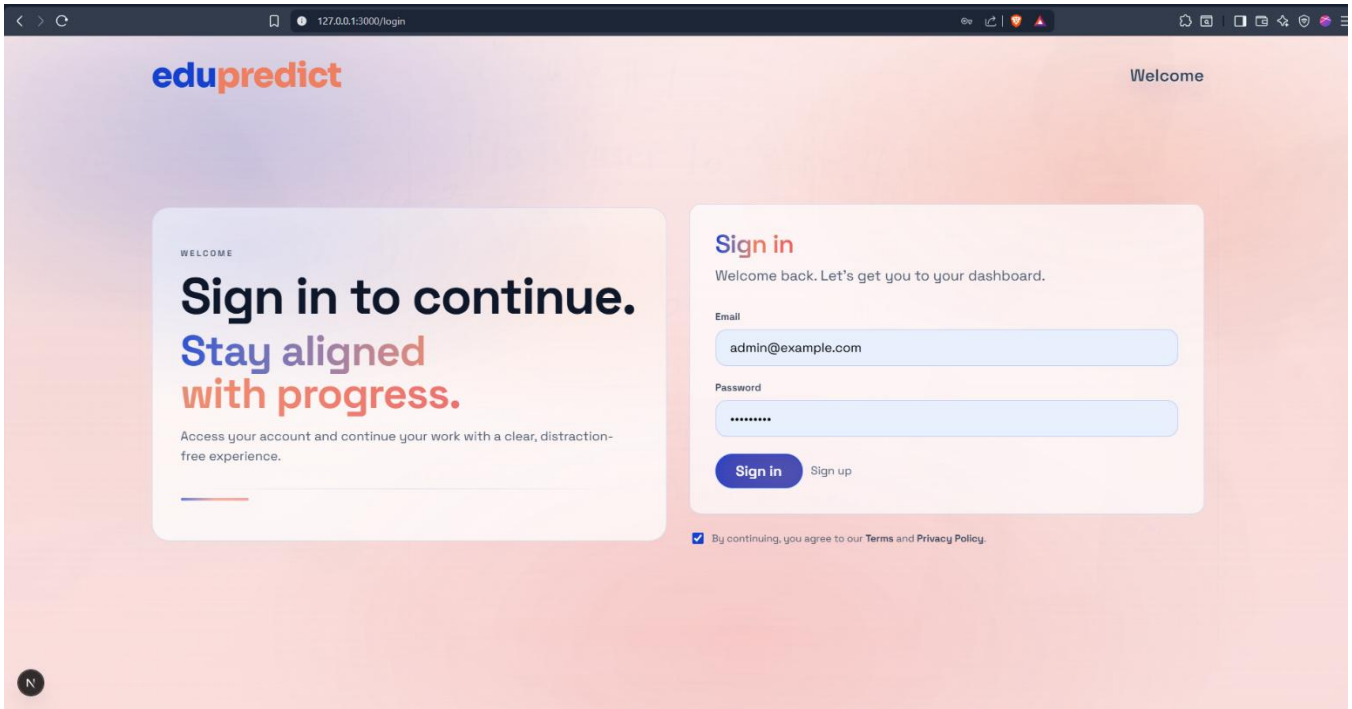
7.5 Screenshots & Execution Results

1) Home Page Of EduPredict

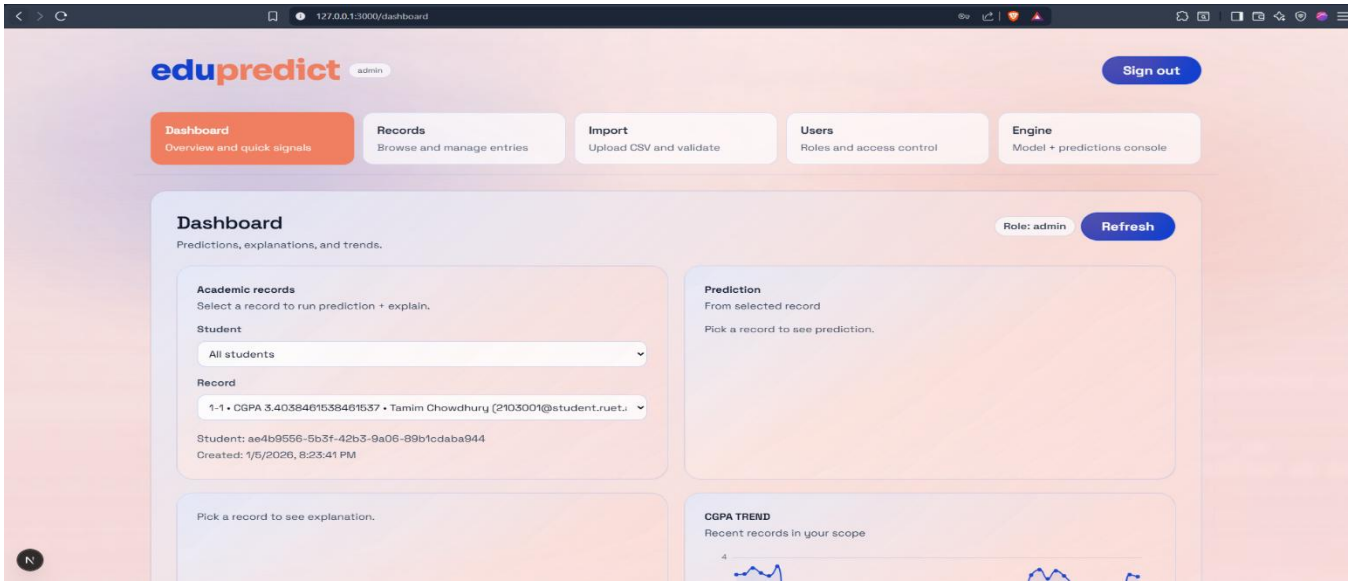


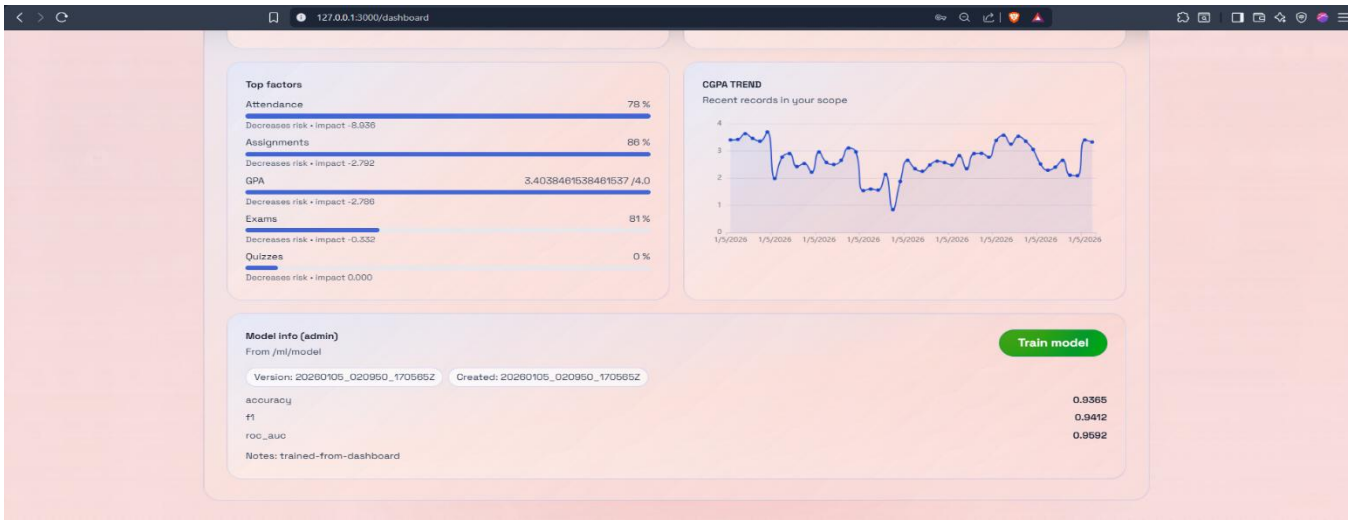
2) Login page and successful authentication

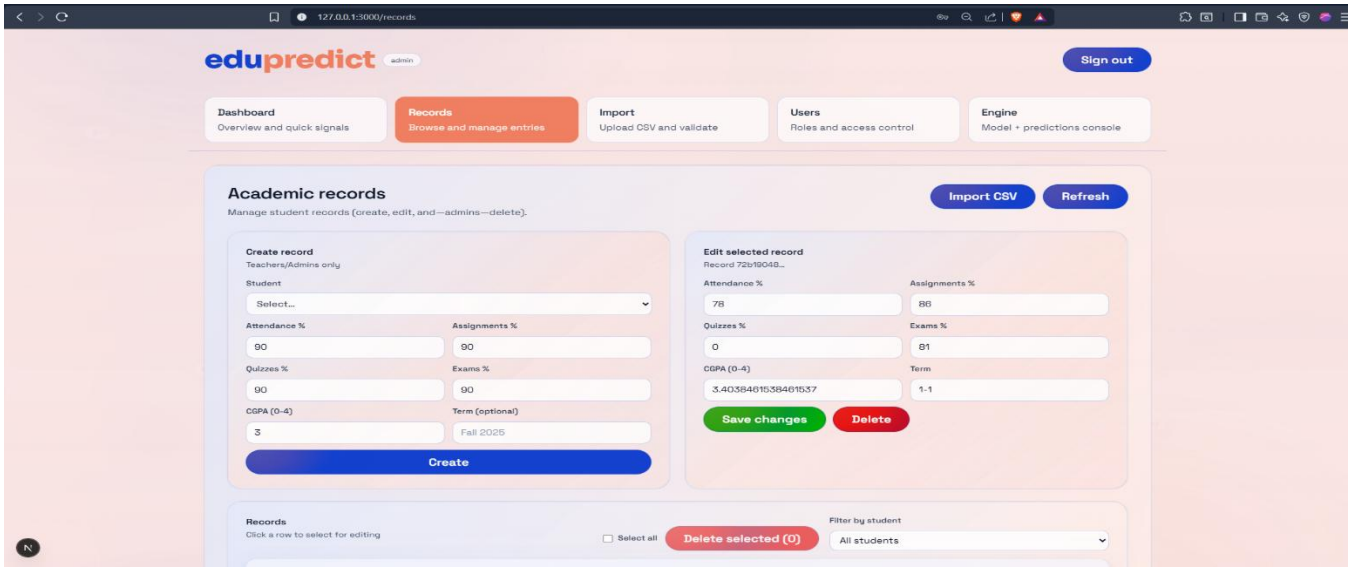




3) Admin dashboard: user list / user creation







127.0.0.1:3000/records

Records
Click a row to select for editing

Select all | Delete selected (0) | Filter by student: All students

Select	Student	CGPA	Term	Created	Signals
	Tamim Chowdhury (2103001@stude...	3.40	1-1	1/5/2026, 8:23:41 PM	78/86/0/81
	Tamim Chowdhury (2103001@stude...	3.42	1-2	1/5/2026, 8:23:41 PM	74/98/0/80
	Tamim Chowdhury (2103001@stude...	3.63	2-1	1/5/2026, 8:23:41 PM	80/94/0/85
	Tamim Chowdhury (2103001@stude...	3.46	2-2	1/5/2026, 8:23:41 PM	79/92/0/81
	Tamim Chowdhury (2103001@stude...	3.36	3-1	1/5/2026, 8:23:41 PM	81/93/0/77
	Tamim Chowdhury (2103001@stude...	3.70	3-2	1/5/2026, 8:23:41 PM	81/91/0/87
	Imran Nashad (2103002@student.ru...	1.98	1-1	1/5/2026, 8:23:41 PM	64/66/0/63
	Imran Nashad (2103002@student.ru...	2.77	1-2	1/5/2026, 8:23:41 PM	62/64/0/70
	Imran Nashad (2103002@student.ru...	2.90	2-1	1/5/2026, 8:23:41 PM	67/69/0/72
	Imran Nashad (2103002@student.ru...	2.42	2-2	1/5/2026, 8:23:41 PM	70/69/0/58
	Imran Nashad (2103002@student.ru...	2.54	3-1	1/5/2026, 8:23:41 PM	70/62/0/63
	Imran Nashad (2103002@student.ru...	2.21	3-2	1/5/2026, 8:23:41 PM	67/65/0/67
	Nafis Khatun (2103003@student.rue...	2.98	1-1	1/5/2026, 8:23:41 PM	68/77/0/72
	Nafis Khatun (2103003@student.rue...	2.58	1-2	1/5/2026, 8:23:41 PM	62/80/0/63
	Nafis Khatun (2103003@student.rue...	2.50	2-1	1/5/2026, 8:23:41 PM	68/73/0/60
	Nafis Khatun (2103003@student.rue...	2.65	2-2	1/5/2026, 8:23:41 PM	65/76/0/63
	Nafis Khatun (2103003@student.rue...	3.11	3-1	1/5/2026, 8:23:41 PM	64/76/0/76
	Nafis Khatun (2103003@student.rue...	2.96	3-2	1/5/2026, 8:23:41 PM	68/80/0/71

127.0.0.1:3000/import

Import academic records

Upload a CSV with either: **summary columns** (attendance_pct, assignments_pct, quizzes_pct, exams_pct, cgpa (column name: gpa), term) or **RUET marks columns** (attendance_10, assignments_10, ct_20, final_60, credits, grade_point_4, semester/term). In both cases provide **student_user_id** or **student_email**.

CSV file

Choose File | course_marks_ruet_cse_50x8.csv

Download template CSV | Excel tip: export as CSV UTF-8 if possible.

☑ Preview only (no changes)

CSV preview | course_marks_ruet_cse_50x8.csv

Detected headers

student_email | student_roll | university | department | semester | course_code | course_title | course_type | credits | attendance_10 | assignments_10 | final_60 | total_80 | letter_grade | grade_point_4

Headers look good.

First rows

student_email	student_roll	university	department	semester	course_code	course_title	course_type
2103001@student.ruet.ac.bd	2103001	RUET	CSE	1-1	1107	Theory Course 1-1 (07)	theory
2103001@student.ruet.ac.bd	2103001	RUET	CSE	1-1	1109	Theory Course 1-1 (09)	theory
2103001@student.ruet.ac.bd	2103001	RUET	CSE	1-1	1102	Sessional/Lab 1-1 (02)	lab
2103001@student.ruet.ac.bd	2103001	RUET	CSE	1-1	1104	Sessional/Lab 1-1 (04)	lab
2103001@student.ruet.ac.bd	2103001	RUET	CSE	1-1	1106	Sessional/Lab 1-1 (06)	lab

Preview shows up to 8 columns and 8 rows.

Check

127.0.0.1:3000/import

eduPredict admin | Sign out

- **Dashboard** — Overview and quick signals
- **Records** — Browse and manage entries
- **Import** — Upload CSV and validate
- **Users** — Roles and access control
- **Engine** — Model + predictions console

Import academic records

Upload a CSV with either: **summary columns** (attendance_pct, assignments_pct, quizzes_pct, exams_pct, cgpa (column name: gpa), term) or **RUET marks columns** (attendance_10, assignments_10, ct_20, final_60, credits, grade_point_4, semester/term). In both cases provide **student_user_id** or **student_email**.

CSV file

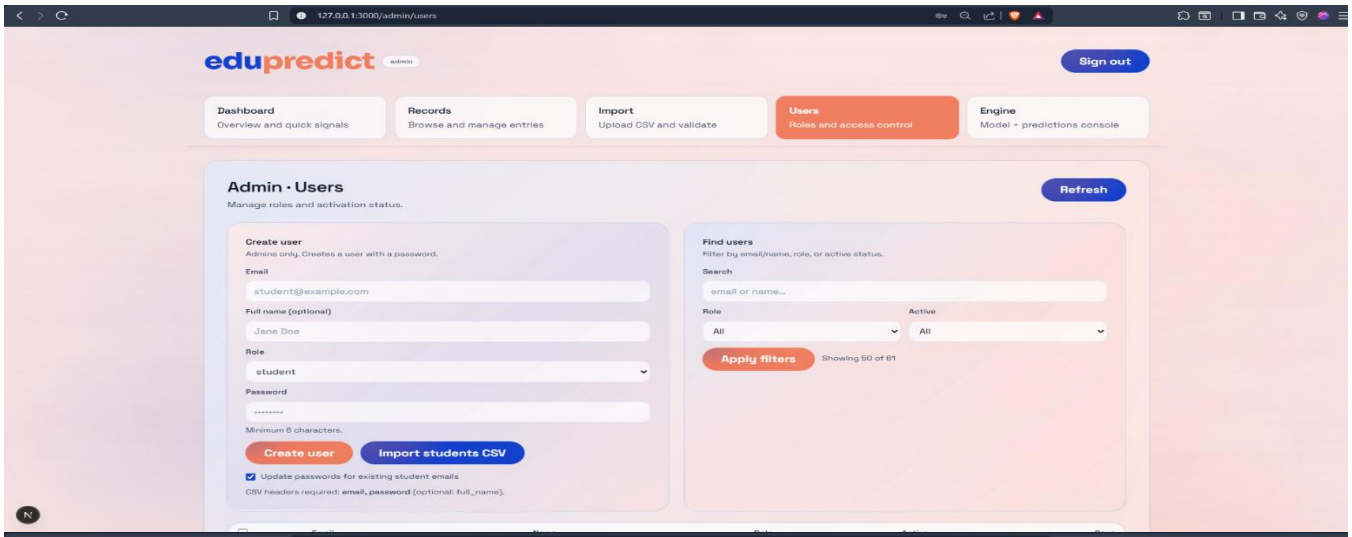
Choose File | No file chosen

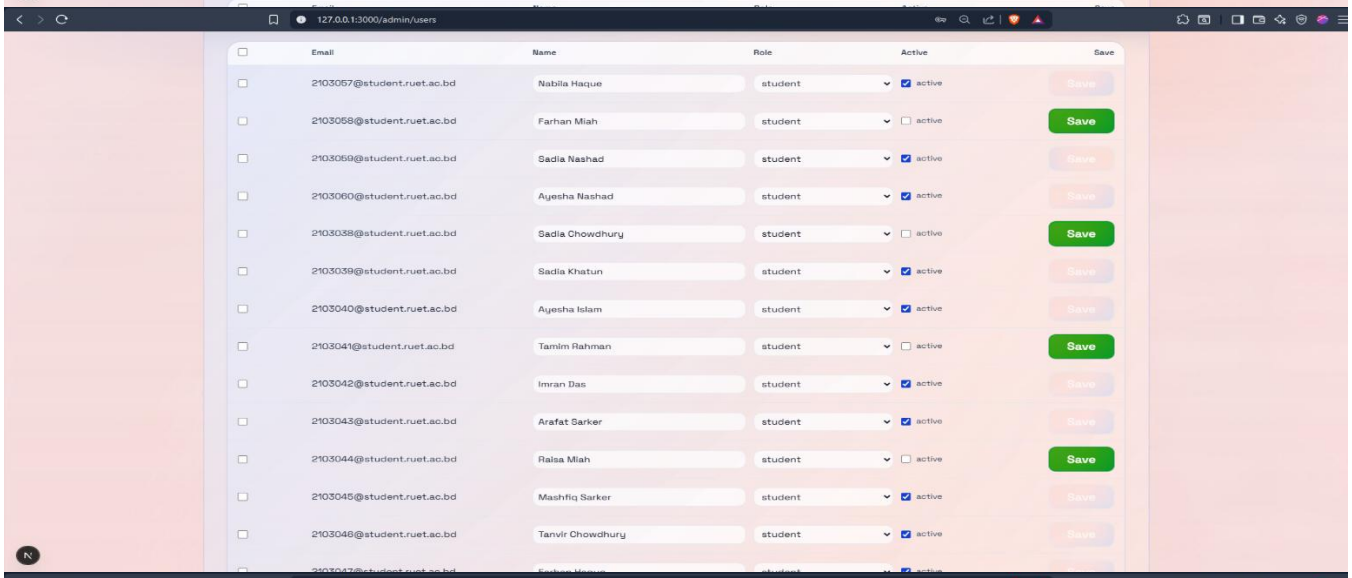
Download template CSV | Excel tip: export as CSV UTF-8 if possible.

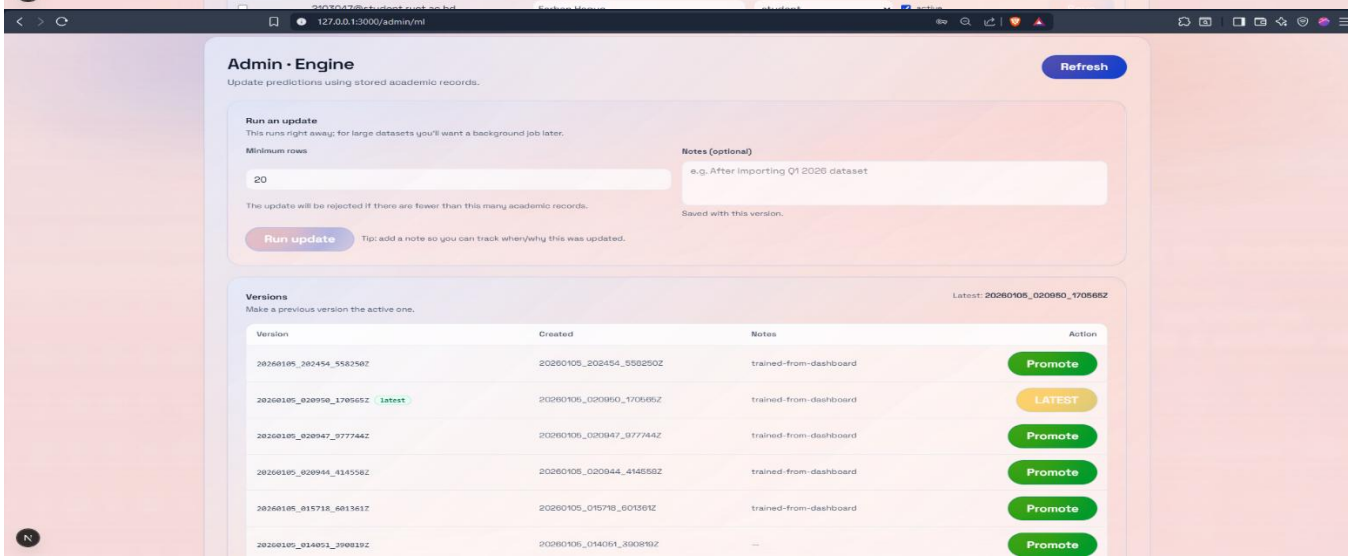
☑ Preview only (no changes)

Check

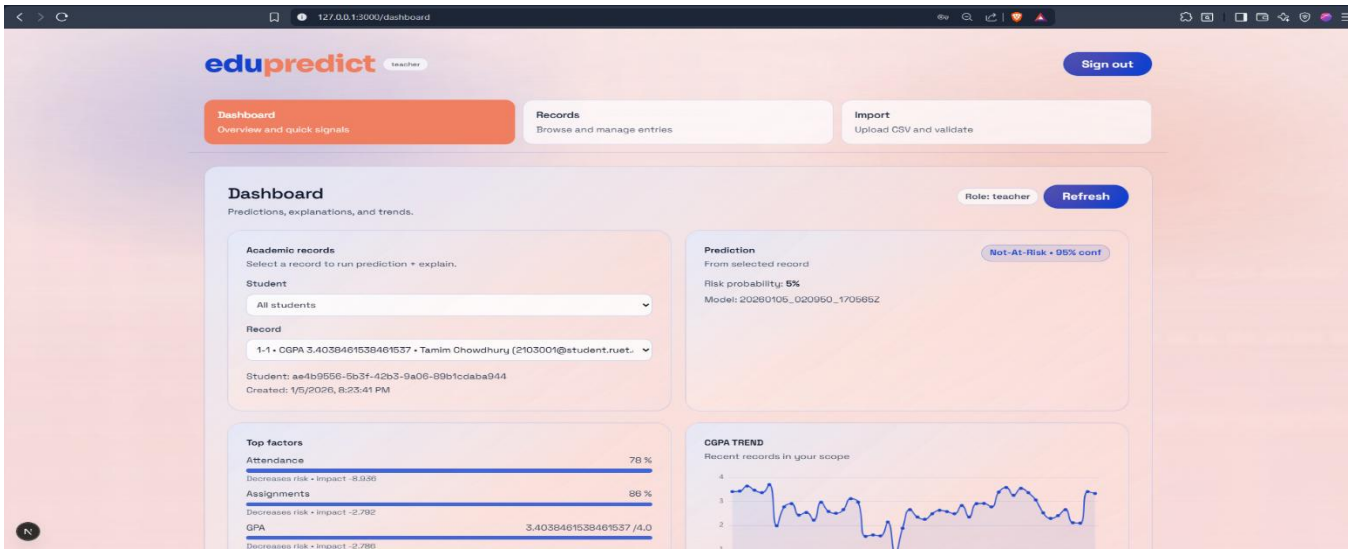
Tip: start with preview to confirm formatting. Turning it off will save the records.

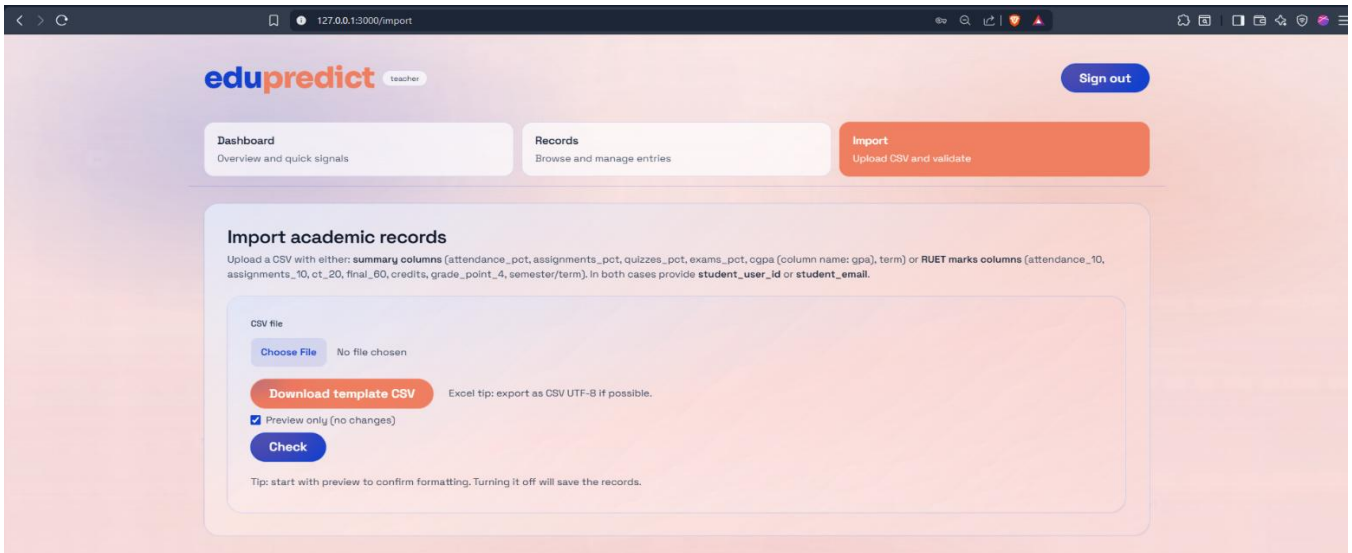

127.0.0.1:3000/admin/users
eduPredict
admin
Sign out
Dashboard
Overview and quick signals
Records
Browse and manage entries
Import
Upload CSV and validate
Users
Roles and access control
Engine
Model + predictions console
Admin · Users
Manage roles and activation status.
Refresh
Create user
Admins only. Creates a user with a password.
Email
student@example.com
Full name (optional)
Jane Doe
Role
student
Password
Minimum 6 characters.
Create user
Import students CSV
Update passwords for existing student emails
CSV headers required: email, password (optional: full_name).
Find users
Filter by email/name, role, or active status.
Search
email or name...
Role
All
Active
All
Apply filters
Showing 50 of 61

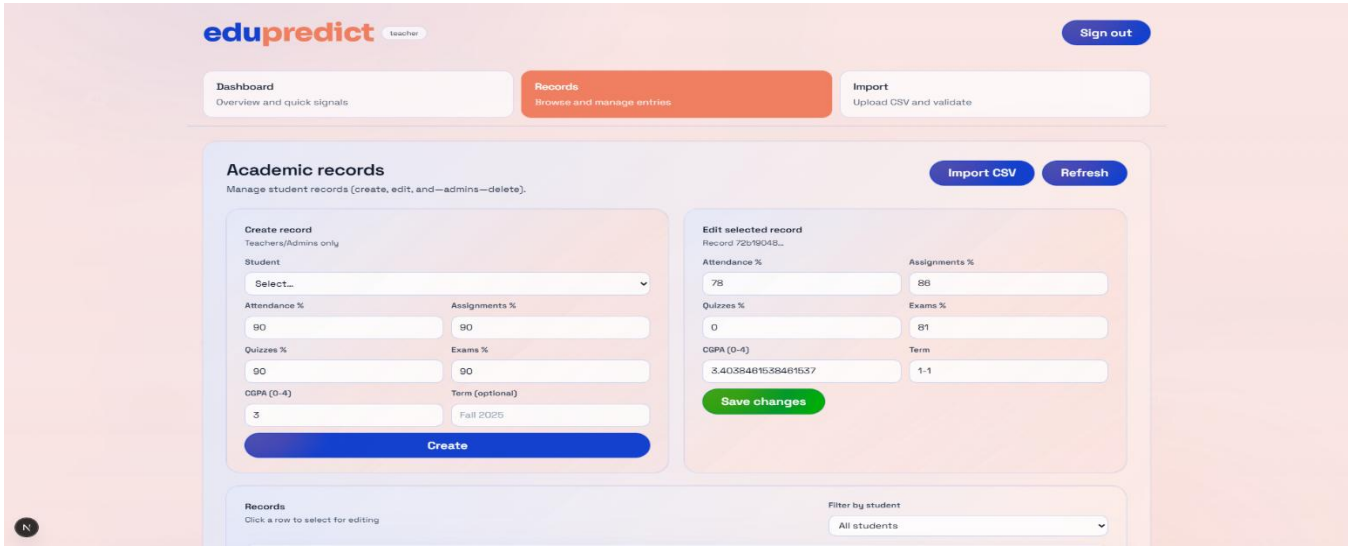

127.0.0.1:3000/admin/users
Email
Name
Role
Active
Save
2103057@student.ruet.ac.bd
Nabila Haque
student
active
2103058@student.ruet.ac.bd
Farhan Miah
student
active
Save
2103059@student.ruet.ac.bd
Sadia Nashad
student
active
2103060@student.ruet.ac.bd
Ayesha Nashad
student
active
2103038@student.ruet.ac.bd
Sadia Chowdhury
student
active
Save
2103039@student.ruet.ac.bd
Sadia Khatun
student
active
2103040@student.ruet.ac.bd
Ayesha Islam
student
active
2103041@student.ruet.ac.bd
Tamim Rahman
student
active
Save
2103042@student.ruet.ac.bd
Imran Das
student
active
2103043@student.ruet.ac.bd
Arafat Sarker
student
active
2103044@student.ruet.ac.bd
Raisa Miah
student
active
Save
2103045@student.ruet.ac.bd
Mashfiq Sarker
student
active
2103046@student.ruet.ac.bd
Tanvir Chowdhury
student
active


127.0.0.1:3000/admin/ml
Admin · Engine
Update predictions using stored academic records.
Refresh
Run an update
This runs right away; for large datasets you'll want a background job later.
Minimum rows
20
The update will be rejected if there are fewer than this many academic records.
Notes (optional)
e.g. After importing Q1 2026 dataset
Saved with this version.
Run update
Tip: add a note so you can track when/why this was updated.
Versions
Make a previous version the active one.
Latest: 20260105_020950_170565Z
Version
Created
Notes
Action
20260105_202454_558250Z
20260105_202454_558250Z
trained-from-dashboard
Promote
20260105_020950_170565Z
latest
20260105_020950_170565Z
trained-from-dashboard
LATEST
20260105_020947_977744Z
20260105_020947_977744Z
trained-from-dashboard
Promote
20260105_020944_414558Z
20260105_020944_414558Z
trained-from-dashboard
Promote
20260105_015718_601361Z
20260105_015718_601361Z
trained-from-dashboard
Promote
20260105_014051_390819Z
20260105_014051_390819Z
—
Promote

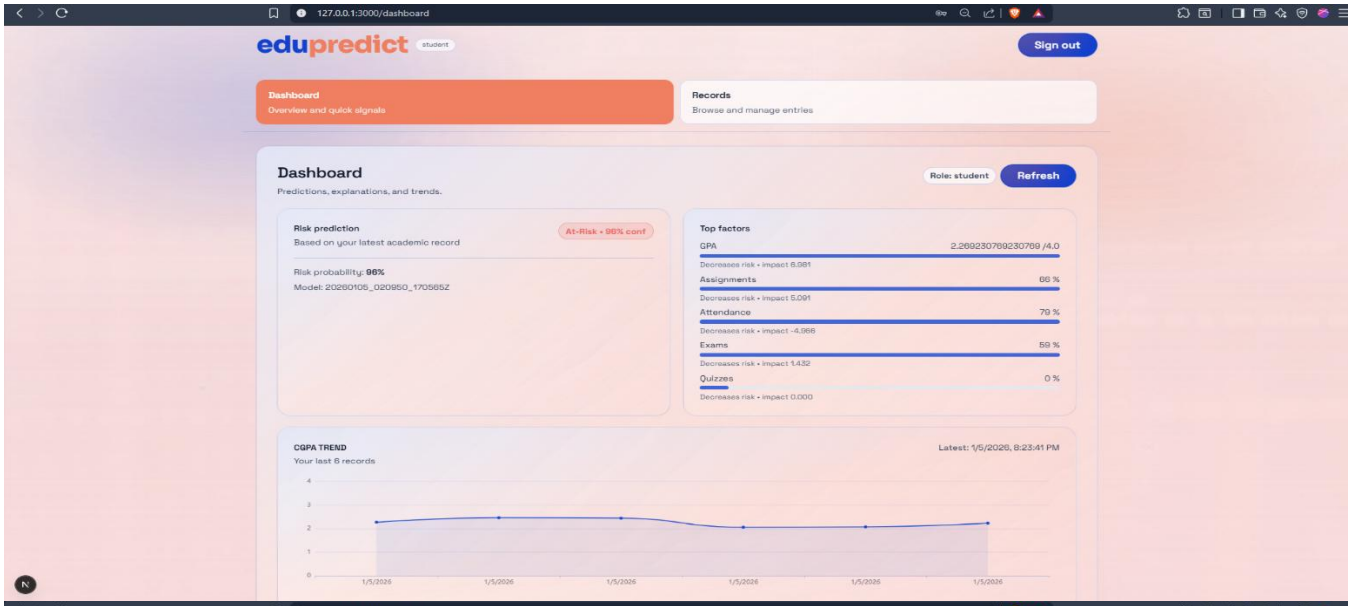
4) Teacher flow: create/update academic record

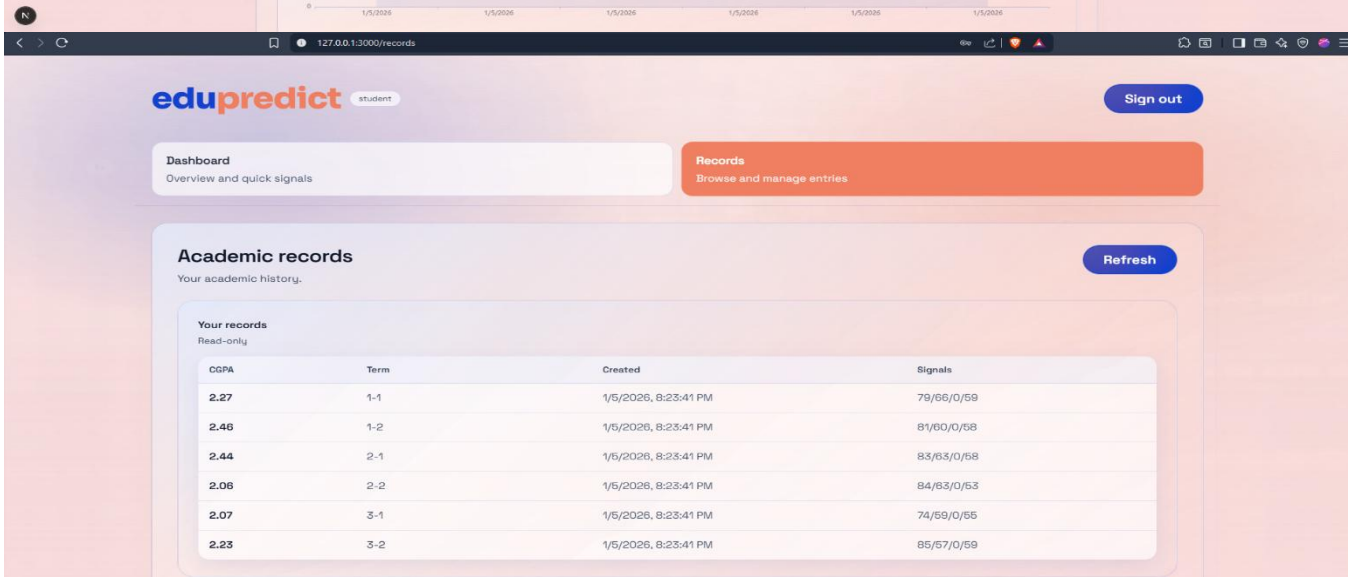




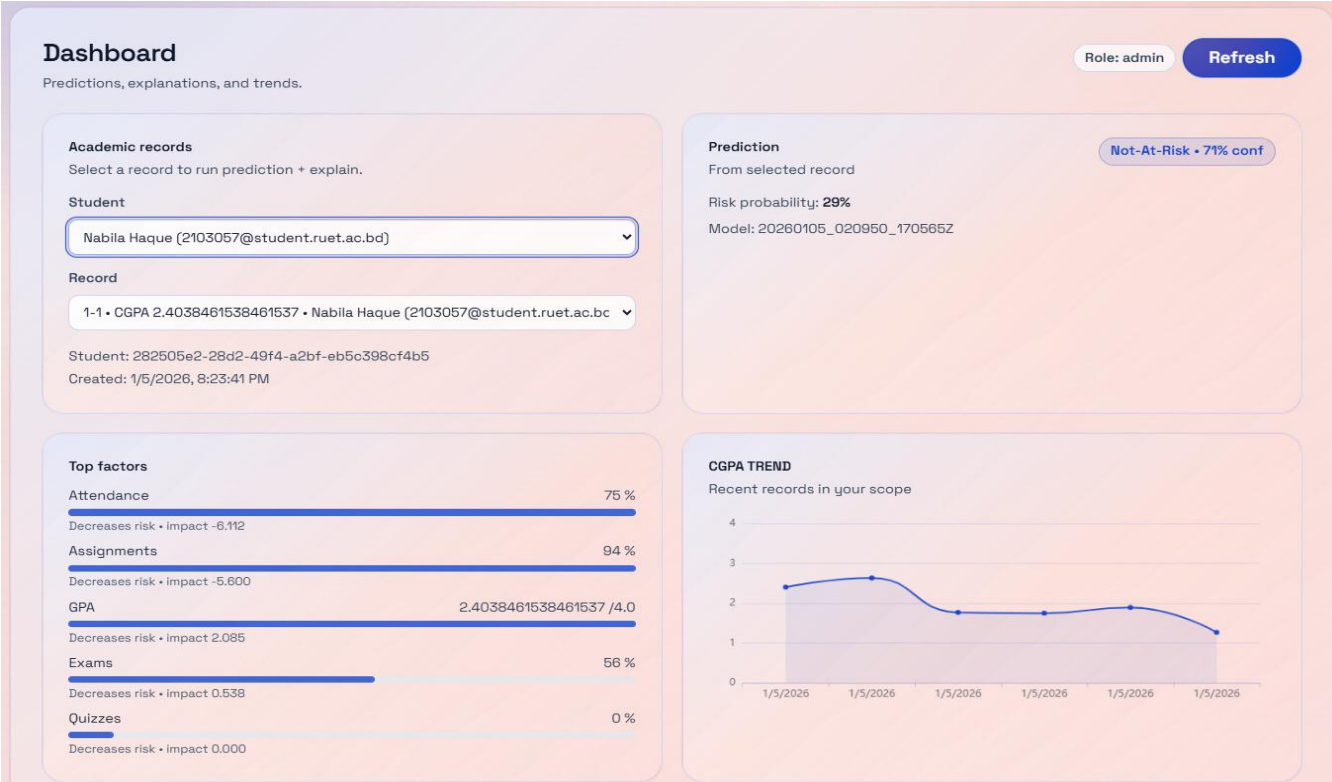


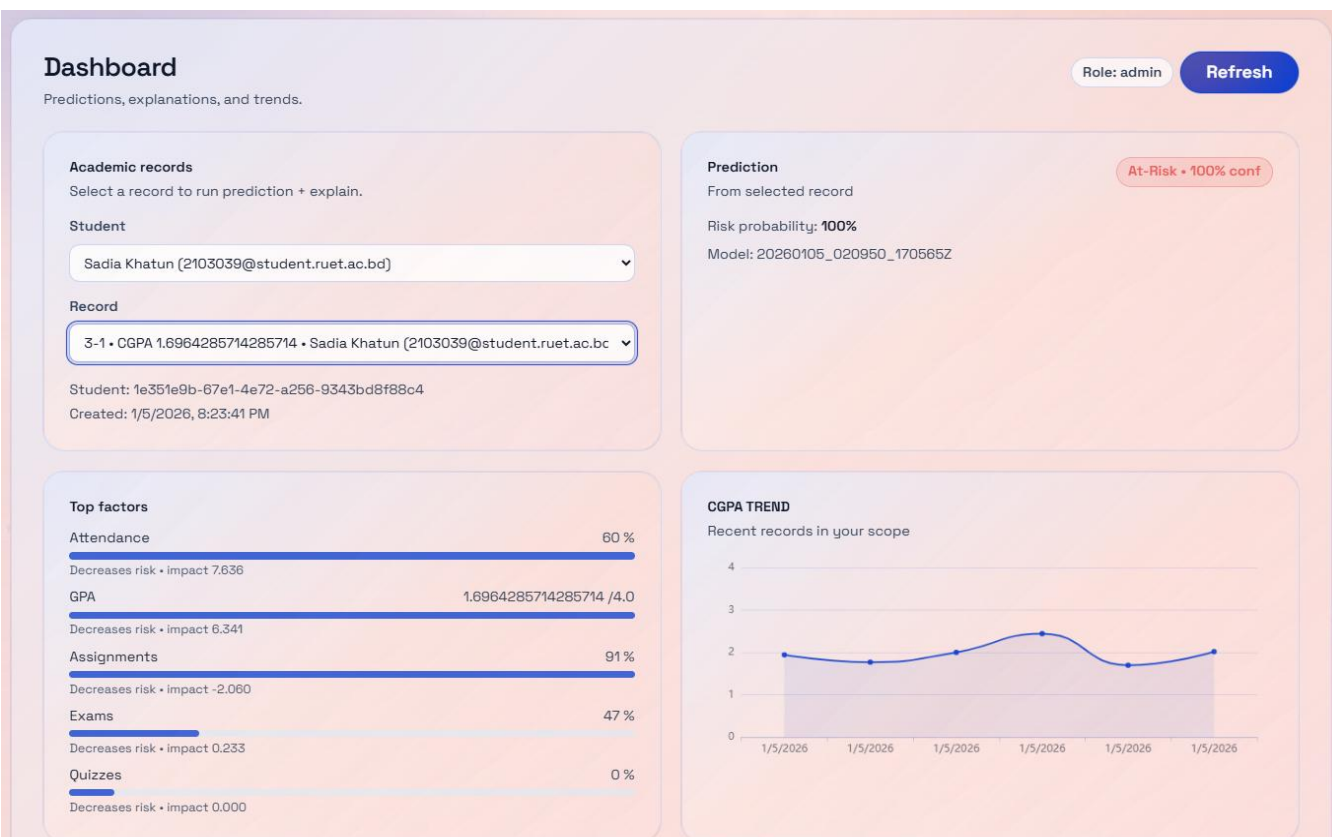
5) Student flow: view own academic records

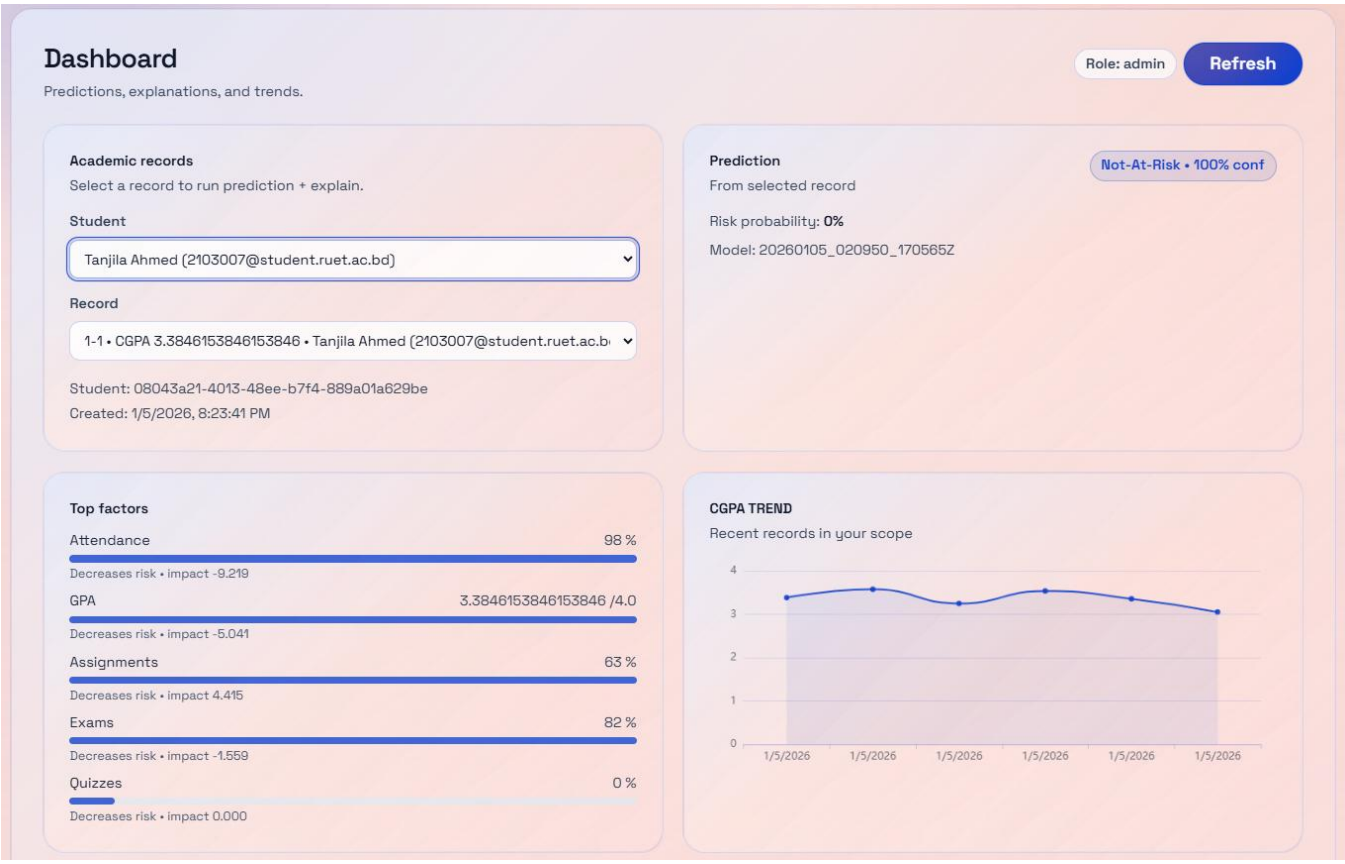




6) ML prediction result (probability + label)






Dashboard
Predictions, explanations, and trends.
Role: admin
Refresh
Academic records
Select a record to run prediction + explain.
Student
Tanjila Ahmed (2103007@student.ruet.ac.bd)
Record
1-1 • CGPA 3.3846153846153846 • Tanjila Ahmed (2103007@student.ruet.ac.b
Student: 08043a21-4013-48ee-b7f4-889a01a629be
Created: 1/5/2026, 8:23:41 PM
Prediction
From selected record
Not-At-Risk • 100% conf
Risk probability: 0%
Model: 20260105_020950_170565Z
Top factors
Attendance 98 %
Decreases risk • impact -9.219
GPA 3.3846153846153846 /4.0
Decreases risk • impact -5.041
Assignments 63 %
Decreases risk • impact 4.415
Exams 82 %
Decreases risk • impact -1.559
Quizzes 0 %
Decreases risk • impact 0.000
CGPA TREND
Recent records in your scope
4
3
2
1
0
1/5/2026
1/5/2026
1/5/2026
1/5/2026
1/5/2026
1/5/2026

Chapter 8
Discussion & Analysis

8.1 Interpretation of Findings

EduPredict produces a probability score that reflects estimated risk rather than a hard decision only. This allows flexible decision-making:

- Higher probability indicates stronger risk signal.
- A configurable threshold converts probability into At-Risk/Not-At-Risk.

The explanation factors identify which inputs contributed most to the risk, enabling targeted interventions:

- *Example:* Low attendance and low exam performance may dominate risk signal.
- Educators can prioritize interventions aligned with those factors.

8.2 Strengths of the Project

- Full end-to-end system (frontend + backend + database + ML + explainability).
- **Strong security implementation:**
    - JWT + refresh tokens
    - Server-side RBAC
    - Student privacy enforcement
- Explainable predictions using SHAP to build trust.
- Model versioning and promotion for controlled updates.
- Local setup is easy with SQLite; Postgres is supported for production-like deployment.

8.3 Limitations

- Model quality depends on data quantity and representativeness.
- Feature set is limited; richer academic and behavioral features could improve accuracy.
- Training is synchronous; large datasets could block the server during training.
- Frontend screenshots and UX evaluation are not included automatically; must be added manually to the report.

8.4 Recommendations for Future Development

- **Add additional features:**
  - Course-wise marks
  - Historical term trends
  - Engagement indicators (LMS activity, assignment submission behavior)
- **Add asynchronous training:**
  - Background job queue (e.g., Celery/RQ) and worker service.
- **Add monitoring:**
  - Model drift detection and performance tracking.
- **Add audit logging:**
  - Record who accessed/modified data (important for institutional compliance).
- **Add richer analytics dashboards:**
  - Cohort trends, risk distribution charts, longitudinal tracking.

8.5 Reflection on Design & Implementation Decisions

Key design decisions were justified as follows:

- Restricting student raw-feature prediction prevents misuse ("gaming") and supports fairness.
- SHAP explanations were chosen to improve interpretability at the cost of extra computation, which is acceptable for single-student analysis.
- Using SQLite by default reduces setup complexity; Postgres support ensures scalability and deployment readiness.
- A file-based model registry was chosen to maintain simplicity and reproducibility without external ML infrastructure.

Chapter 9
Life-long Learning Impact

9.1 Technical & Professional Skills Acquired

- API design and implementation using FastAPI.
- Secure authentication and session handling (JWT, refresh tokens, revocation).
- RBAC design for real-world multi-role systems.
- Database modeling and migrations using SQLAlchemy and Alembic.
- Integrating ML pipelines into production-style backends.
- Understanding explainability methods (SHAP) and presenting interpretable outputs.
- Testing async systems using pytest-asyncio.
- Documentation and deployment planning for full-stack applications.

9.2 Learning New Technologies & Tools

- **Next.js App Router** architecture and modern frontend patterns.
- **TypeScript-based** frontend development for safer UI code.
- **Tailwind CSS** utility-first responsive design.
- **LightGBM** for efficient gradient-boosted modeling on tabular data.
- **SHAP TreeExplainer** usage and handling binary-class explanation outputs.
- **Docker Compose** for local service orchestration with Postgres.

9.3 Future Growth & Directions

- Extending EduPredict into a full institutional early warning system.
- Improving dataset quality and evaluation methodology for more reliable predictions.
- Learning production MLOps practices: CI/CD, monitoring, drift detection, scheduled retraining.
- Applying similar secure + explainable ML architecture to other domains (health risk prediction, dropout prediction, etc.).

Chapter 10
Conclusion

10.1 Summary of Work

EduPredict delivers a secure, role-based student performance monitoring platform that predicts academic risk using machine learning and provides human-interpretable explanations. The system includes academic record management, authentication and RBAC enforcement, model training and registry management, and a modern frontend for stakeholders.

10.2 Achievement of Objectives

- **Full-stack platform implemented (Frontend + Backend):** Achieved
- **JWT authentication with refresh tokens:** Achieved
- **Server-side RBAC for Admin/Teacher/Student:** Achieved
- **Academic record CRUD with correct permissions:** Achieved
- **ML training, prediction, and explainability endpoints:** Achieved
- **Model registry with versioning and promotion:** Achieved
- **Local setup support (SQLite) and optional Postgres via Docker:** Achieved
- **Automated backend tests validating key flows:** Achieved

10.3 Final Remarks

EduPredict demonstrates how software engineering and explainable machine learning can be combined to support proactive academic intervention. The project provides a practical, deployable foundation for early risk identification and can be extended with richer datasets, asynchronous training, and monitoring features to become more robust for institutional use.

References

1. S. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," NeurIPS, 2017.
2. LightGBM: A Highly Efficient Gradient Boosting Decision Tree, Microsoft/LightGBM project documentation.
3. scikit-learn: Machine Learning in Python, scikit-learn documentation.
4. FastAPI Documentation, FastAPI project.
5. SQLAlchemy Documentation, SQLAlchemy project.
6. Alembic Documentation, Alembic project.

Appendices

Appendix A: Technology Stack Summary

- **Frontend:** Next.js (App Router) + TypeScript + Tailwind CSS
- **Backend:** FastAPI + SQLAlchemy (async) + Alembic
- **ML:** scikit-learn + LightGBM + SHAP
- **Database:** SQLite (default) / Postgres (Docker option)

Appendix B: Key Modules (Repository Structure)

- main.py — FastAPI app creation and router mounting
- apps/backend/app/routers/ — auth, admin, academics, ml, health, bootstrap
- apps/backend/app/models/ — users, refresh tokens, academic records
- apps/backend/app/ml/ — training, inference, explainability, registry, preprocess
- apps/backend/alembic/versions/ — DB migration scripts
- apps/frontend/ — Next.js frontend source and configuration

Appendix C: Suggested Screenshot Checklist

- Cover page + certification page
- Swagger UI (/docs)
- Login success + role-based dashboard
- Academic record creation/update
- Student record view (self-only)
- Prediction output screen
- Explanation output screen (top-k factors)
- Admin user list + create/update user
- Admin model list + promote model (if UI/API demonstrated)